Program no. 9

Using Flex and Bison tools create a program to convert
(a) An infix arithmetic expression into Postfix expression.

infix_postfix.l:

```
%{
#include "infix_postfix.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+            { yylval.str = strdup(yytext); return NUMBER; }
[\+\-\*/\^]       { return *yytext; }
[\(\)]            { return *yytext; }
[ \t\n]           ;   // Ignore whitespace
.                 { return yytext[0]; }
%%

int yywrap() { return 1; }
```

infix_postfix.y:

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);
```

```
// Helper macro to replace asprintf
#define MAKE_STR(result, fmt, ...)                  \
    do {                                            \
        size_t len = snprintf(NULL, 0, fmt, __VA_ARGS__); \
        result = malloc(len + 1);                   \
        sprintf(result, fmt, __VA_ARGS__);          \
    } while(0)
%}

%union {
    char *str;
}

%token <str> NUMBER
%left '+' '-'
%left '*' '/'
%right '^'

%type <str> expr

%%

input:
    expr { printf("Postfix: %s\n", $1); free($1); }
    ;

expr:
    expr '+' expr { MAKE_STR($$, "%s %s +", $1, $3); free($1); free($3); }
  | expr '-' expr { MAKE_STR($$, "%s %s -", $1, $3); free($1); free($3); }
  | expr '*' expr { MAKE_STR($$, "%s %s *", $1, $3); free($1); free($3); }
  | expr '/' expr { MAKE_STR($$, "%s %s /", $1, $3); free($1); free($3); }
```

```
  | expr '^' expr { MAKE_STR($$, "%s %s ^", $1, $3); free($1); free($3); }
  | '(' expr ')'  { $$ = $2; }
  | NUMBER       { $$ = strdup($1); free($1); }
  ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}
int main() {
    return yyparse();
}
```

Output:

```
F:\Flex\win_flex>win_bison -d infix_postfix.y

F:\Flex\win_flex>win_flex infix_postfix.l

F:\Flex\win_flex>gcc infix_postfix.tab.c lex.yy.c -o infix_postfix.exe -lm -mconsole

F:\Flex\win_flex>.\infix_postfix.exe
(3 + 4) * 5


Postfix: 3 4 + 5 *
```

(b) An infix arithmetic expression into Prefix expression.

infix_prefix.l:

```
%{
#include "infix_prefix.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+            { yylval.str = strdup(yytext); return NUMBER; }
[\+\-\*/\^]       { return *yytext; }
[\(\)]            { return *yytext; }
```

```
[ \t\n]           ;   // Ignore whitespace
.                 { return yytext[0]; }
%%

int yywrap() { return 1; }
```

infix_prefix.y:

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);

#define MAKE_STR(result, fmt, ...)                  \
   do {                                             \
      size_t len = snprintf(NULL, 0, fmt, __VA_ARGS__); \
      result = malloc(len + 1);                     \
      sprintf(result, fmt, __VA_ARGS__);            \
   } while(0)
%}

%union {
   char *str;
}

%token <str> NUMBER
%left '+' '-'
%left '*' '/'
%right '^'
```

```
%type <str> expr

%%

input:
    expr { printf("Prefix: %s\n", $1); free($1); }
    ;

expr:
    expr '+' expr { MAKE_STR($$, "+ %s %s", $1, $3); free($1); free($3); }
  | expr '-' expr { MAKE_STR($$, "- %s %s", $1, $3); free($1); free($3); }
  | expr '*' expr { MAKE_STR($$, "* %s %s", $1, $3); free($1); free($3); }
  | expr '/' expr { MAKE_STR($$, "/ %s %s", $1, $3); free($1); free($3); }
  | expr '^' expr { MAKE_STR($$, "^ %s %s", $1, $3); free($1); free($3); }
  | '(' expr ')'  { $$ = $2; }
  | NUMBER        { $$ = strdup($1); free($1); }
  ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    return yyparse();
}
```

Output:

```
F:\Flex\win_flex>win_bison -d infix_prefix.y

F:\Flex\win_flex>win_flex infix_prefix.l

F:\Flex\win_flex>gcc infix_prefix.tab.c lex.yy.c -o infix_prefix.exe -lm

F:\Flex\win_flex>infix_prefix.exe
(3 + 4) * 5
Prefix: * + 3 4 5
```

(c) An Prefix arithmetic expression into Postfix expression.

prefix_postfix.l:

```
%{

#include "prefix_postfix.tab.h"

#include <stdlib.h>

%}


%%

"+"     { return PLUS; }

"-"     { return MINUS; }

"*"     { return TIMES; }

"/"     { return DIVIDE; }

"^"     { return POWER; }

[0-9]+  { yylval.str = strdup(yytext); return NUMBER; }

[ \t\n]  ;  // Skip whitespace

.       { return yytext[0]; }

%%


int yywrap() { return 1; }
```

prefix_postfix.y:

```
%{

#include <stdio.h>

#include <string.h>

#include <stdlib.h>
```

```
void yyerror(const char *s);
int yylex(void);

#define MAKE_STR(result, fmt, ...)                  \
   do {                                             \
      size_t len = snprintf(NULL, 0, fmt, __VA_ARGS__); \
      result = malloc(len + 1);                     \
      sprintf(result, fmt, __VA_ARGS__);            \
   } while(0)
%}

%union {
   char *str;
}

%token <str> NUMBER
%token PLUS MINUS TIMES DIVIDE POWER
%type <str> expr

%%

input:
   expr { printf("Postfix: %s\n", $1); free($1); }
   ;

expr:
   PLUS expr expr   { MAKE_STR($$, "%s %s +", $2, $3); free($2); free($3); }
 | MINUS expr expr  { MAKE_STR($$, "%s %s -", $2, $3); free($2); free($3); }
 | TIMES expr expr  { MAKE_STR($$, "%s %s *", $2, $3); free($2); free($3); }
 | DIVIDE expr expr { MAKE_STR($$, "%s %s /", $2, $3); free($2); free($3); }
 | POWER expr expr  { MAKE_STR($$, "%s %s ^", $2, $3); free($2); free($3); }
```

```
  | NUMBER          { $$ = strdup($1); free($1); }
  ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    return yyparse();
}
```

Output:

```
F:\Flex\win_flex>win_bison -d prefix_postfix.y

F:\Flex\win_flex>win_flex prefix_postfix.l

F:\Flex\win_flex>gcc prefix_postfix.tab.c lex.yy.c -o prefix_postfix.exe -lm

F:\Flex\win_flex>prefix_postfix.exe
* + 3 4 5
Postfix: 3 4 + 5 *
```